

JAVA PROGRAMMING

MODULE 3

EXCEPTION HANDLING

What is exception?

- Unexpected and unwanted situation in the program execution is called exception.
- Exception will disturb normal flow of the program execution.
- When exception occurred program will be terminated abnormally.
- Note- as a programmer we are responsible for programs graceful termination.
- To achieve graceful termination we need to handle the exceptions occurred while program executing.
- The process of handling exception is called as exception handling.
- The main aim of exception handling to achieve graceful termination of the program\ In java we have so many predefined exceptions.

AirithmeticException

NullPointerException

FileNotFoundException

SQLException

DEALING WITH ERRORS

Introduction to Errors

In Java, errors are issues or problems that can interrupt the normal flow of a program. These problems can occur at compile-time, run-time, or due to incorrect logic written by the programmer. Java provides a strong mechanism called exception handling to manage such errors during program execution. It helps in writing robust programs that don't crash unexpectedly and can handle errors gracefully.

1. Compile-Time Errors

Compile-time errors in Java are those that are identified by the compiler when the program is being compiled, before it runs. These errors typically occur due to violations of the syntax rules defined by the Java language. Some common examples include missing semicolons, incorrect variable declarations, incompatible data types, undeclared variables, or calling methods with wrong parameters. For instance, trying to assign a string to an integer variable (`int x = "hello";`) would result in a compile-time error because the data types are incompatible. Similarly, forgetting to close a brace or using a keyword incorrectly also causes such errors. These must be corrected in the code before the program can be successfully compiled and executed. IDEs

like Eclipse or IntelliJ help by highlighting these errors, making it easier for developers to fix them quickly.

2. Runtime Errors (Exceptions)

Runtime errors are errors that occur while the program is running, after it has successfully compiled. These errors are typically caused by operations that are valid in syntax but invalid during execution, such as dividing a number by zero, trying to access an array element that doesn't exist, or calling a method on a null object. Since the compiler cannot detect these errors, they are known as exceptions, and Java provides a robust mechanism to handle them through exception handling using try, catch, and finally blocks. For example, if a program tries to divide a number by zero (`int result = 10 / 0;`), it will throw an `ArithmeticException` at runtime, causing the program to terminate unless the error is caught and handled. Other common runtime errors include `NullPointerException`, `ArrayIndexOutOfBoundsException`, and `ClassCastException`. These can be avoided or properly handled using safe coding practices and error-handling structures.

2. Runtime Errors (Exceptions)

Runtime errors are errors that occur while the program is running, after it has successfully compiled. These errors are typically caused by operations that are valid in syntax but invalid during execution, such as dividing a number by zero, trying to access an array element that doesn't exist, or calling a method on a null object. Since the compiler cannot detect these errors, they are known as exceptions, and Java provides a robust mechanism to handle them through exception handling using try, catch, and finally blocks. For example, if a program tries to divide a number by zero (`int result = 10 / 0;`), it will throw an `ArithmeticException` at runtime, causing the program to terminate unless the error is caught and handled. Other common runtime errors include `NullPointerException`, `ArrayIndexOutOfBoundsException`, and `ClassCastException`. These can be avoided or properly handled using safe coding practices and error-handling structures.

3. Logical Errors

Logical errors are the most difficult type of errors to detect because the program compiles and runs without crashing but produces incorrect or unexpected results. These errors occur due to mistakes in the program's logic or algorithm, which leads to incorrect output even though the syntax is correct. For example, if a program is supposed to calculate the average of three numbers but mistakenly divides their sum by two instead of three, the logic is flawed. Logical errors often result from incorrect use of conditional statements, loops, formulas, or calculations. Since the compiler and runtime environment do not report these errors, it's the programmer's responsibility to thoroughly test and debug the application to identify and fix logical flaws. Using proper testing methods and code reviews helps in minimizing these errors.

Java provides a try-catch-finally mechanism to handle exceptions.

Syntax of Exception Handling

```
try
{
```

```
        // Code that may throw an exception
    }

    catch (ExceptionType e)
    {

        // Handle the exception

    }

    finally

    {

        // This block always executes (optional)

    }
```

Benefits of Exception Handling in Java

1. Prevents Program Crashes

One of the primary benefits of exception handling in Java is that it prevents the program from crashing unexpectedly. When an error occurs during program execution, Java's exception handling mechanism allows the developer to catch and manage the error gracefully using try and catch blocks. This ensures that the application can either recover from the error or exit in a controlled manner, rather than stopping abruptly with a system-generated error message.

2. Improves Program Reliability

By properly handling exceptions, Java programs become more reliable and stable. Even if something goes wrong during execution, such as a missing file or an invalid input, the program can continue running or take an alternative course of action. This increases the reliability of the software, especially in real-world applications where unexpected input or system failures are common.

3. Separates Error Handling from Main Logic

Java's exception-handling structure allows developers to separate normal business logic from error-handling code. This makes the main code cleaner and easier to read, as all the logic for handling errors is placed in separate catch or finally blocks. This separation improves code clarity and makes maintenance easier since the logic and error handling are clearly distinguished.

4. Provides Specific Error Messages

Exception handling allows developers to catch specific types of errors and provide meaningful, custom error messages. Instead of showing confusing system errors to the user, the program can display friendly messages that indicate what went wrong. This not only improves user experience but also helps developers identify and fix bugs more quickly.

5. Supports Custom Exceptions

Java allows the creation of user-defined or custom exceptions. This means developers can define their own exception classes to handle application-specific problems in a more descriptive and manageable way. Custom exceptions help make the code more understandable and maintainable, especially in large projects where generic exception messages are not helpful.

6. Ensures Resource Management

The finally block in Java ensures that important tasks, such as closing files, releasing database connections, or cleaning up memory, are performed no matter what. Whether an exception occurs or not, the code in the finally block is always executed. This ensures that resources are not wasted or left open, which is important for efficient and error-free programs.

7. Encourages Robust and Secure Code

By anticipating and handling all potential runtime errors, exception handling helps write programs that are robust and secure. It prevents the application from behaving unpredictably in the presence of errors, and reduces the chances of security risks or data loss caused by unhandled exceptions.

8. Allows Grouping of Error Types

Java supports multiple catch blocks, allowing developers to handle different types of exceptions separately. This makes it easier to write specific responses for different error conditions. For example, `ArithmeticException` can be handled differently than `FileNotFoundException`, providing better control and organization.

9. Promotes Better User Experience

When exceptions are handled properly, users are not faced with technical error messages or system crashes. Instead, they receive clear and friendly messages or alternative options. This leads to a smoother and more professional user experience, especially in user-facing applications.

10. Supports Exception Propagation

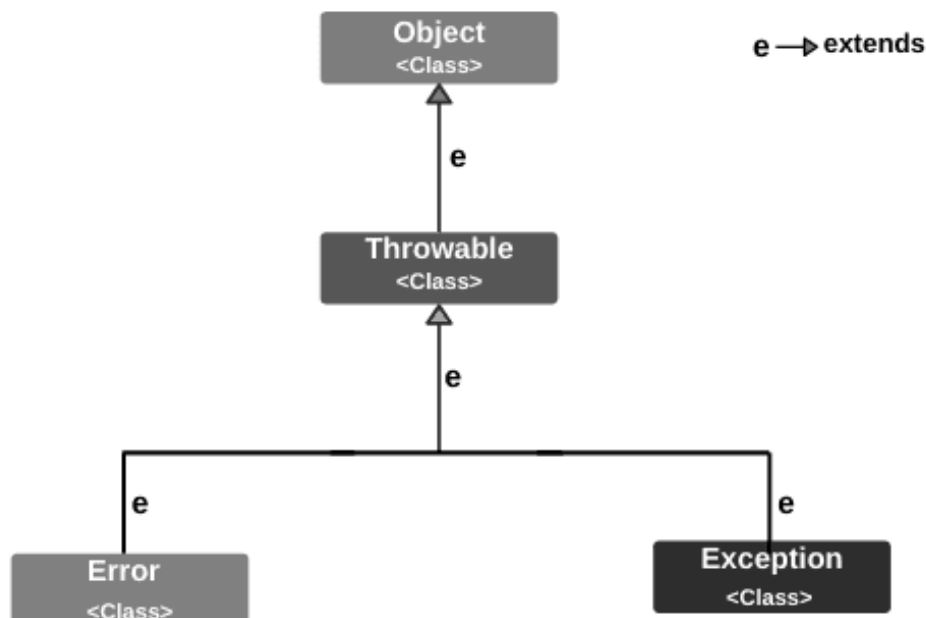
Java provides the `throws` keyword, which allows a method to declare the exceptions it might throw. This feature is useful for passing exception responsibility to another method or layer in the program, often where a centralized error-handling mechanism exists. It simplifies error management in large applications and avoids redundant code.

Classification of Exceptions

Predefined exceptions are those exceptions that are already defined by Java system. These exceptions are also called built-in-exceptions. Java API supports exception handling by providing the number of predefined exceptions. These predefined exceptions are represented by classes in Java.

When a predefined exception occurs, JVM (Java runtime system) creates an object of predefined exception class. All exceptions are derived from java.lang.Throwable class but not all exception classes are defined in the same package. All the predefined exceptions supported by Java are organized as subclasses in a hierarchy under the Throwable class.

In Java programming, Throwable class is the root of exception hierarchy and is an immediate subclass of Object class. Let's understand the Java exception hierarchy, as shown in the below figure.



1. Throwable class: As shown in the above figure, Throwable class which is derived from Object class, is a top of exception hierarchy from which all exception classes are derived directly or indirectly. It is the root of all exception classes. It is present in java.lang package. Throwable class is the superclass of all exceptions in java. This class has two subclasses: Error and Exception. Errors or exceptions occurring in java programs are objects of these classes. Using Throwable class, you can also create your own custom exceptions.

2. Error: Error class is the subclass of Throwable class and a superclass of all the runtime error classes. It terminates the program if there is problem-related to a system or resources (JVM). An error generally represents an unusual problem or situation from which it is difficult to recover. It does not occur by programmer mistakes. It generally occurs if the system is not working properly or resource is not allocated properly.

VirtualMachineError, StackOverflowError, AssertionError, LinkageError, OutOfMemoryError, etc are examples of error. We will learn more detail in further tutorials.

3. Exception: It is represented by an Exception class that represents errors caused by the program and by external factors. Exception class is a subclass of Throwable class and a superclass of all the exception classes. All the exception classes are derived directly or indirectly from the Exception class. They originate from within the application.

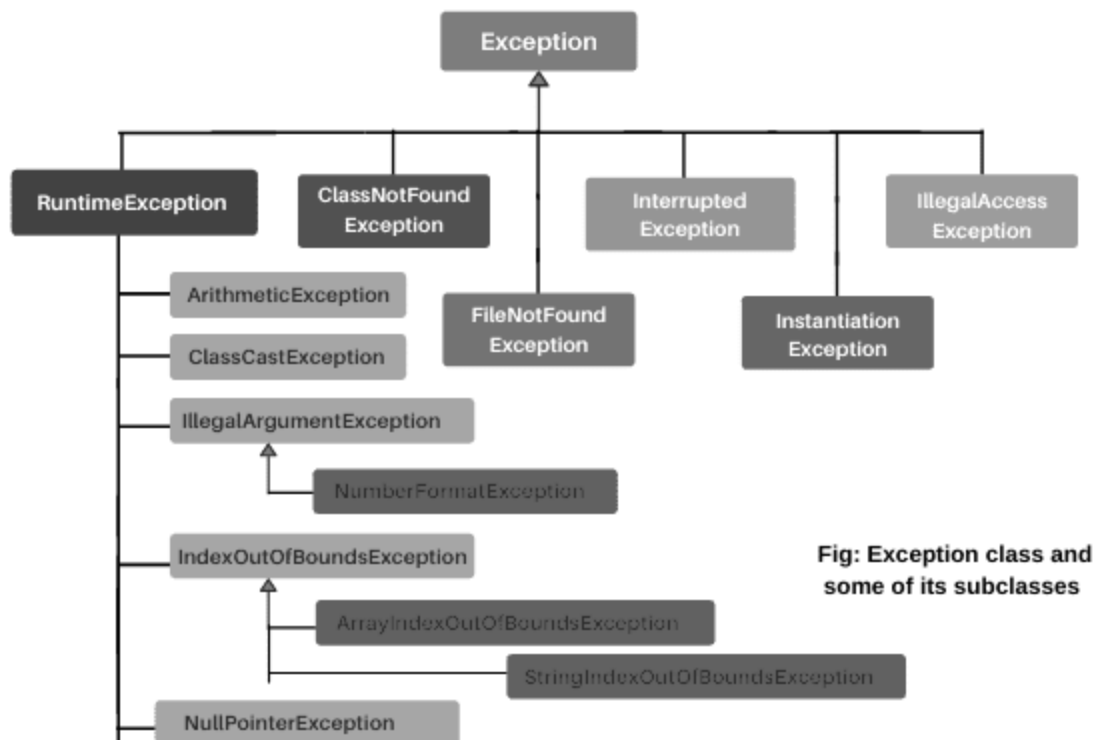
The exception class provides two constructors:

- `public Exception()` (Default constructor)
- `public Exception(String message)` (It takes a message string as argument)

Each of the exception classes provides two constructors: one with no argument and another with a String type argument. Exception class does not provide its own method. It inherits all methods provided by Throwable class.

Exception Class Hierarchy in Java

We can see the hierarchy of exception class in Java in the below figure that is very important for an interview purpose.



Custom Exceptions

Custom exceptions are those exceptions that are created by users or programmers according to their own needs. The custom exceptions are also called user-defined exceptions that are created by extending the exception class.

So, Java provides the liberty to programmers to throw and handle exceptions while dealing with functional requirements of problems they are solving.

```

class MyException extends Exception
{
    public MyException(String message)

```

```

{
    super(message);
}
}

public class CustomExceptionExample
{
    public static void main(String[] args)
    {
        try
        {
            throw new MyException("Custom Exception Occurred!");
        }

        catch (MyException e)
        {
            System.out.println(e.getMessage());
        }
    }
}

```

Let's see the brief description of each subclass of the Exception class.

RuntimeException Class (Unchecked Exceptions)

RuntimeException class is a subclass of the Exception class. It is thrown by JVM or programmatically when an arithmetic operation performed in the program is incorrect or defect/bug occurs in the program's code.

Java compiler does not check RuntimeException and all its exception subclasses because they occur during runtime of a program. That's why these exceptions are also called unchecked exceptions. RuntimeException class consists of many other exception subclasses that are used to handle a specific type of exception. Apart from these exception subclasses of RuntimeException class shown in the above figure. There are other subclasses of

RuntimeException class, which have not been shown in the hierarchy structure diagram to avoid complexity.

Let's see a brief description of them.

1. ArithmeticException:

This exception is thrown when arithmetic problems, such as a number is divided by zero, is occurred. That is, it is caused by maths error.

```
public class Example1

{

    public static void main(String[] args)

    {

        int a = 20;

        int b = 0;

        int result = a / b; // This will cause ArithmeticException

        System.out.println("Result: " + result);

    }

}
```

Output

Exception in thread "main" java.lang.ArithmeticException: / by zero

2. ClassCastException:

The ClassCastException is a runtime exception that is thrown by JVM when we attempt to invalid typecasting in the program. That is, it is thrown when we cast an object to a subclass of which an object is not an instance.

```
public class Example2

{

    public static void main(String[] args)

    {

        Object obj = new String("Hello World");
```



```

        try
        {
            Integer num = (Integer) obj; // Invalid cast

            System.out.println("Number: " + num);

        }

        catch (ClassCastException e)
        {

            System.out.println("Error: Cannot cast String to Integer.");

        }

        System.out.println("Program continues after handling the exception.");

    }

}

```

Output:

Error: Cannot cast String to Integer.

Program continues after handling the exception.

3. **IllegalArgumentException:**

This runtime exception is thrown by programmatically when an illegal or appropriate argument is passed to call a method. This exception class has further two subclasses:

1.NumberFormatException

2.IllegalThreadStateException

```

public class Example2 {

    public static void main(String[] args) {

        int age = -10;

        try

        {

            setAge(age);

```

```

    }

    catch (IllegalArgumentException e)

    {

        System.out.println("Error: " + e.getMessage());

    }

    System.out.println("Program continues after handling the exception.");

}

public static void setAge(int age) {

    if (age < 0) {

        throw new IllegalArgumentException("Age cannot be negative.");

    }

    System.out.println("Age is: " + age);

}

}

```

Output:

Error: Age cannot be negative.

Program continues after handling the exception.

1. **NumericFormatException**: **NumberFormatException** is thrown by programmatically when we try to convert a string into the numeric type and the process of illegal conversion fails. That is, it occurs due to the illegal conversion of a string to a numeric format.

2. **IllegalThreadStateException**: **IllegalThreadStateException** exception is a runtime exception that is thrown by programmatically when we attempt to perform any operation on a thread but it is incompatible with the current thread state.

4. **IndexOutOfBoundsException**:

This exception class is thrown by JVM when an array or string is going out of the specified index. It has two further subclasses:

1. **ArrayIndexOutOfBoundsException**
2. **StringIndexOutOfBoundsException**

1. **ArrayIndexOutOfBoundsException**: `ArrayIndexOutOfBoundsException` exception is thrown when an array element is accessed out of the index.

2. **StringIndexOutOfBoundsException**: `StringIndexOutOfBoundsException` exception is thrown when a `String` or `StringBuffer` element is accessed out of the index.

```
public class Example
```

```
{  
  
    public static void main(String[] args)  
  
    {  
  
        int[] numbers = {10, 20, 30};  
  
        // Trying to access index 3 which is out of bounds (valid indexes: 0, 1, 2)  
  
        System.out.println("Value: " + numbers[3]);  
  
    }  
}
```

Output:

Exception in thread "main" java.lang.ArrayIndexOutOfBoundsException: Index 3 out of bounds for length 3

5. **NullPointerException**:

`NullPointerException` is a runtime exception that is thrown by JVM when we attempt to use null instead of an object. That is, it is thrown when the reference is null.

```
public class Example
```

```
{  
  
    public static void main(String[] args)  
  
    {  
  
        String str = null; // str is not pointing to any object  
  
        // Trying to call a method on a null object  
  
        System.out.println("Length of string: " + str.length());  
  
    }  
}
```

Output:

Exception in thread "main" java.lang.NullPointerException: Cannot invoke "String.length()" because "str" is null

5. ArrayStoreException:

This exception occurs when we attempt to store any value in an array which is not of array type. For example, suppose, an array is of integer type but we are trying to store a value of an element of another type.

```
public class Example {  
  
    public static void main(String[] args) {  
  
        String str = null; // str is not pointing to any object  
  
        // Trying to call a method on a null object  
  
        System.out.println("Length of string: " + str.length());  
  
    }  
  
}
```

Output:

Exception in thread "main" java.lang.NullPointerException: Cannot invoke "String.length()" because "str" is null

Checked and Unchecked Exceptions in Java

We know that there are two types of exceptions in Java: first is predefined exceptions, and second user-defined exceptions.

The predefined exceptions are those exceptions that are already defined by the java system. All the predefined exceptions are further divided into two groups:

- Checked Exceptions
- Unchecked Exceptions

Checked Exceptions in Java

Checked exceptions are those exceptions that are checked by the Java compiler itself at compilation time and are not under runtime exception class hierarchy. If a method throws a checked exception in a program, the method must either handle the exception or pass it to a caller method.

We must handle checked exceptions either by using try and catch block or by using a throws clause in the method declaration. If not handles properly, it will give a compile-time error.

Most people have confusion and say that checked exceptions occur at compile-time, that is wrong. All exceptions always occur at runtime only, but some exceptions are detected at compile-time and some other at runtime.

The exceptions that are checked by Java compiler at compilation time are called checked exceptions in Java. All the exceptions except RuntimeException, Error, and their subclasses are checked exceptions.

List of Checked Exceptions in Java

A list of some important checked exceptions is given below:

- ClassNotFoundException
- InterruptedException
- InstantiationException
- IOException
- SQLException
- IllegalAccessException
- FileNotFoundException, etc

Unchecked Exceptions (Runtime Exceptions) in Java

Unchecked exceptions in Java are those exceptions that are checked by JVM, not by java compiler. They occur during the runtime of a program. All exceptions under the runtime exception class are called unchecked exceptions or runtime exceptions in Java.

We can write a Java program and compile it. But we cannot see the effect of unchecked exceptions and errors until we run the program. This is because Java compiler allows us to write a Java program without handling unchecked exceptions and errors.

Java compiler does not check runtime exception at compile time whether programmer handles them or not. If a runtime exception occurs in a method and the programmer does not handle it, JVM terminates the program without the execution of rest of the code.

List of Unchecked Exceptions in Java

Some important examples of runtime exceptions are given below:

- ArithmeticException
- ClassCastException

- NullPointerException
- ArrayIndexOutOfBoundsException
- NegativeArraySizeException
- ArrayStoreException
- IllegalThreadStateException
- SecurityException, etc.

Usage of try, catch, throw, throws and finally

Exception handling in Java is a mechanism to handle runtime errors, preventing the program from crashing. Java provides five main keywords for handling exceptions: try, catch, throw, throws, and finally. Each of these plays a crucial role in making Java programs more robust and error-free.

1. try block

The try block is used to wrap the code that might throw an exception. It defines a block of statements where exceptions may occur. If an exception occurs inside the try block, the control immediately jumps to the corresponding catch block. Without a try block, exceptions can cause the program to terminate abruptly. For example, if we attempt to divide a number by zero inside a try block, the program will look for an appropriate catch block to handle the error instead of terminating unexpectedly.

```
public class TryExample
{
    public static void main(String[] args) {
        try {
            int result = 10 / 0; // This will cause an ArithmeticException
            System.out.println(result);
        }
    }
}
```

catch block

2. catch block

The catch block follows the try block and is used to handle the exceptions thrown within the try block. It specifies the type of exception it can catch and provides an alternative execution flow when an error occurs. If an exception occurs in the try block and a corresponding catch block is present, the program executes the catch block instead of crashing. For instance, if an ArithmeticException occurs due to division by zero, the catch block can display a meaningful message like "Cannot divide by zero" instead of allowing the program to terminate.

```

public class CatchExample
{
    public static void main(String[] args)
    {
        try
        {
            int result = 10 / 0;
        } catch (ArithmeticException e)
        {
            System.out.println("Cannot divide by zero.");
        }
    }
}

```

3. throw

The throw keyword is used to manually throw an exception within the program. It allows programmers to create their own custom error messages and exceptions instead of relying only on predefined Java exceptions. For example, if a method checks the age of a voter and finds that the age is below 18, it can use the throw keyword to generate an ArithmeticException with a message stating that the person is not eligible to vote.

```

public class ThrowExample
{
    static void checkAge(int age)
    {
        if (age < 18) {
            throw new ArithmeticException("Not eligible to vote");
        } else {
            System.out.println("Eligible to vote");
        }
    }
}

```

```

    }

    public static void main(String[] args) {

        checkAge(16); // This will throw an exception

    }

}

```

Output: Exception in thread "main" java.lang.ArithmeticException: Not eligible to vote

4. throws

The throws keyword is used in method signatures to declare exceptions that might be thrown by a method. Instead of handling the exception within the method, throws informs the caller that an exception may occur, leaving it to the caller to handle the exception appropriately. This is particularly useful when dealing with checked exceptions, such as file handling or database connections, where a method might fail and needs to inform the calling function about the potential error.

```

public class ThrowsExample {

    static void divide() throws ArithmeticException {

        int num = 10 / 0; // May throw an exception

    }

    public static void main(String[] args) {

        try {

            divide();

        } catch (ArithmeticException e) {

            System.out.println("Exception caught: " + e);

        }

    }

}

```

Output: Exception caught: java.lang.ArithmeticException: / by zero

5. finally

The finally block is an optional block that always executes, regardless of whether an exception occurs or not. It is mainly used for cleanup activities, such as closing files, releasing resources, or ending database connections. Even if an exception occurs and is caught, or if the program runs successfully without exceptions, the finally block will execute. This ensures that critical operations, like freeing memory or closing network connections, are performed properly.

```
public class FinallyExample {  
  
    public static void main(String[] args) {  
  
        try {  
  
            int num = 10 / 0;  
  
        } catch (ArithmeticException e) {  
  
            System.out.println("Exception caught.");  
  
        } finally {  
  
            System.out.println("Finally block executed.");  
  
        }  
  
    }  
  
}
```

Output:

Exception caught.

Finally block executed.

Multithreading in java

The process of executing multiple tasks (also called threads) simultaneously is called multithreading. The primary purpose of multithreading is to provide simultaneous execution of two or more parts of a program to make maximum use of CPU time. A multithreaded program contains two or more parts that can run concurrently. It enables programmers to write in a way where multiple activities can proceed simultaneously within a single application.

It is the way of executing multiple tasks at a time executing them concurrently over a specified period. Multitasking is done in two ways. These are:

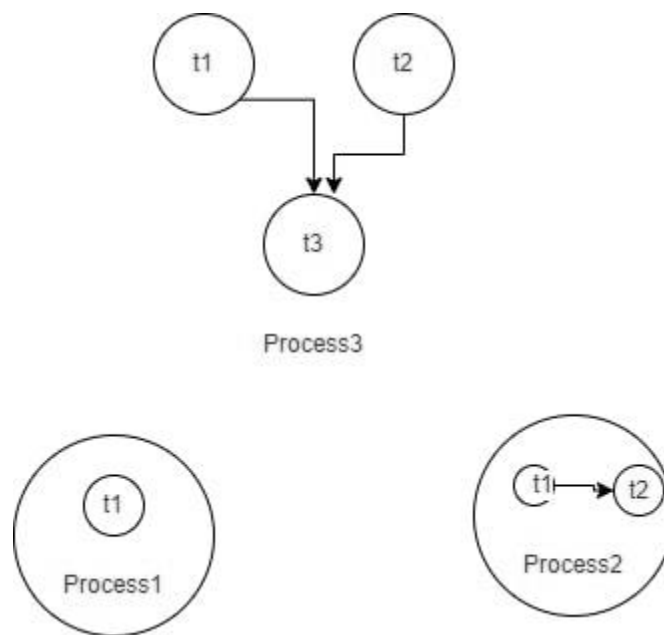
- **Process-based multitasking:** It is also called multiprocessing where each process has its address in memory, i.e., each process allocates separate memory area.

- Thread-based multitasking: This thread-based multitasking is also termed as multithreading where threads share the same address space.

Java Threads

Threads are lightweight subprocesses, representing the smallest unit of execution with separate paths. The main advantage of multiple threads is efficiency (allowing multiple things at the same time). For example, in MS Word, one thread automatically formats the document while another thread is taking user input. Additionally, multithreading ensures quick response, as other threads can continue execution even if one gets stuck, keeping the application responsive.

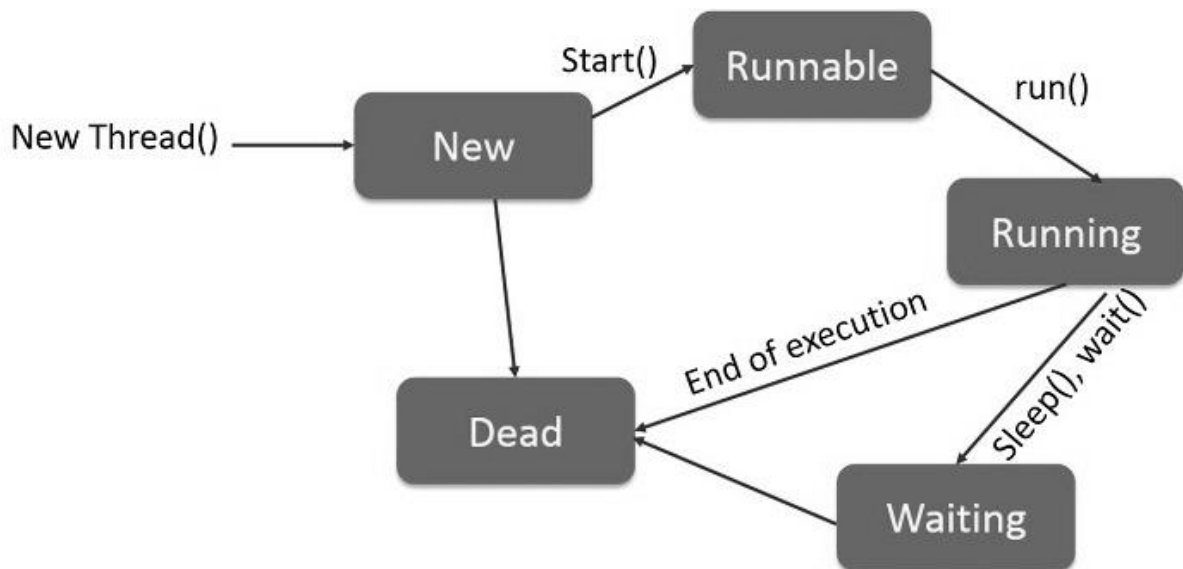
Threads in a Shared Memory Environment in OS



As we can observe in, the above diagram a thread runs inside the process and there will be context-based switching between threads there can be multiple processes running in OS, and each process again can have multiple threads running simultaneously. The Multithreading concept is popularly applied in games, animation...etc.

Life Cycle of a Thread in Java

The life cycle of a thread in Java refers to the various states of a thread goes through. For example, a thread is born, started, runs, and then dies. Thread class defines the life cycle and various states of a thread.



States of a Thread Life Cycle in Java

- ❖ New State
- ❖ Active State
- ❖ Waiting/Blocked State
- ❖ Timed Waiting State
- ❖ Terminated State

Following are the stages of the life cycle –

1. New State: By default, a Thread will be in a new state, in this state, code has not yet been run and the execution process is not yet initiated.

2. Active State: A Thread that is in a new state by default gets transferred to Active state when it invokes the start() method, this Active state contains two sub-states namely:

Runnable State: In This State, The Thread is ready to run at any given time and it's the job of the Thread Scheduler to provide the thread time for the runnable state preserved threads. A program that has obtained Multithreading shares slices of time intervals which are shared between threads hence, these threads run for some short span of time and wait in the runnable state to get their scheduled slice of a time interval.

Running State: When The Thread Receives CPU allocated by Thread Scheduler, it transfers from the "Runnable" state to the "Running" state. and after the expiry of its given time slice session, it again moves back to the "Runnable" state and waits for its next time slice.

3. Waiting/Blocked State: If a Thread is inactive but on a temporary time, then either it is a waiting or blocked state, for example, if there are two threads, T1 and T2 where T1 needs to communicate to the camera and the other thread T2 already using a camera to scan then T1 waits until T2 Thread completes its work, at this state T1 is parked in waiting for the state, and in another scenario, the user called two Threads T2 and T3 with the same functionality and both had same time slice given by Thread Scheduler then both Threads T1, T2 is in a blocked state. When there are multiple threads parked in a Blocked/Waiting state Thread Scheduler clears Queue by rejecting unwanted Threads and allocating CPU on a priority basis.

4. Timed Waiting State: Sometimes the longer duration of waiting for threads causes starvation, if we take an example like there are two threads T1, T2 waiting for CPU and T1 is undergoing a Critical Coding operation and if it does not exist the CPU until its operation gets executed then T2 will be exposed to longer waiting with undetermined certainty, In order to avoid this starvation situation, we had Timed Waiting for the state to avoid that kind of scenario as in Timed Waiting, each thread has a time period for which sleep() method is invoked and after the time expires the Threads starts executing its task.

5. Terminated State: A thread will be in Terminated State, due to the below reasons:

Termination is achieved by a Thread when it finishes its task Normally.

Sometimes Threads may be terminated due to unusual events like segmentation faults, exceptions...etc. and such kind of Termination can be called Abnormal Termination.

A terminated Thread means it is dead and no longer available. Java Example for Demonstrating Thread States

In this example, we're creating two threads by extending the Thread class. We're printing each state of the thread. When a thread object is created, its state is NEW; when start() method is called, state is START; when run() method is called, state is RUNNING; When a thread finished the processing the run() method, it went to DEAD state.

```
package java;
```

```
class ThreadDemo extends Thread {
```

```
    private Thread t;
```

```
    private String threadName;
```

```
    ThreadDemo( String name) {
```

```
        threadName = name;
```

```
        System.out.println("Thread: " + threadName + ", " + "State: New");
```

```
    }
```

```
    public void run() {
```

```
        System.out.println("Thread: " + threadName + ", " + "State: Running");
```

```
        for(int i = 4; i > 0; i--) {
```

```
            System.out.println("Thread: " + threadName + ", " + i);
```

```
        }
```

```
        System.out.println("Thread: " + threadName + ", " + "State: Dead");
```

```
    }
```

```

public void start () {

    System.out.println("Thread: " + threadName + ", " + "State: Start");

    if (t == null) {

        t = new Thread (this, threadName);

        t.start ();

    }

}

}

public class TestThread {

    public static void main(String args[]) {

        ThreadDemo thread1 = new ThreadDemo( "Thread-1");

        ThreadDemo thread2 = new ThreadDemo( "Thread-2");

        thread1.start();

        thread2.start();

    }

}

```

Output:

Thread: Thread-1, State: New

Thread: Thread-2, State: New

Thread: Thread-1, State: Start

Thread: Thread-2, State: Start

Thread: Thread-1, State: Running

Thread: Thread-2, State: Running

Thread: Thread-2, 4

Thread: Thread-2, 3

Thread: Thread-2, 2

Thread: Thread-2, 1

Thread: Thread-2, State: Dead

Thread: Thread-1, 4

Thread: Thread-1, 3

Thread: Thread-1, 2

Thread: Thread-1, 1

Thread: Thread-1, State: Dead

Thread Methods in Java

Java provides a built-in Thread class in the java.lang package that helps in multithreading, allowing multiple tasks to run concurrently. The Thread class provides several methods to control thread execution.

1. start() Method

The start() method is used to start a new thread. It calls the run() method internally and allows the thread to execute separately from the main thread.

Example:

```
class MyThread extends Thread {  
    public void run() {  
        System.out.println("Thread is running...");  
    }  
}  
  
public class StartExample {  
    public static void main(String[] args) {  
        MyThread t1 = new MyThread();  
        t1.start(); // Starts the thread  
    }  
}
```

Output:

Thread is running...

2. run() Method

The run() method contains the code that will be executed when the thread starts. It must be overridden in a custom thread class.

Example:

```
class MyThread extends Thread {  
  
    public void run() {  
  
        System.out.println("Run method executed...");  
  
    }  
  
}  
  
public class RunExample {  
  
    public static void main(String[] args) {  
  
        MyThread t1 = new MyThread();  
  
        t1.run(); // Calls run() like a normal method, not a separate thread  
  
    }  
  
}
```

Output:

Run method executed...

3. sleep(ms) Method

The sleep() method pauses the execution of the current thread for a specified time (in milliseconds). It throws InterruptedException if another thread interrupts it while sleeping.

Example:

```
class SleepExample extends Thread {  
  
    public void run() {  
  
        for (int i = 1; i <= 5; i++) {  
  
            try {  
  
                Thread.sleep(1000); // Pause for 1 second
```

```

        } catch (InterruptedException e) {

            System.out.println(e);

        }

        System.out.println(i);

    }

}

public static void main(String args[]) {

    SleepExample t1 = new SleepExample();

    t1.start();

}

}

```

Output (prints numbers with a 1-second delay):

```

1
2
3
4
5

```

4. join() Method

The join() method ensures that a thread completes execution before another thread starts.

Example:

```

class JoinExample extends Thread {

    public void run() {

        for (int i = 1; i <= 5; i++) {

            System.out.println(Thread.currentThread().getName() + " - " + i);

        }

    }

}

```



```

    }

    public static void main(String args[]) {

        JoinExample t1 = new JoinExample();

        JoinExample t2 = new JoinExample();

        t1.start();

        try {

            t1.join(); // Ensures t1 finishes before t2 starts

        } catch (InterruptedException e) {

            System.out.println(e);

        }

        t2.start();

    }

}

```

Output (t1 runs completely before t2 starts):

Thread-0 - 1

Thread-0 - 2

Thread-0 - 3

Thread-0 - 4

Thread-0 - 5

Thread-1 - 1

Thread-1 - 2

Thread-1 - 3

Thread-1 - 4

Thread-1 - 5

5. isAlive() Method

The `isAlive()` method checks if a thread is still running or has finished execution.

Example:

```
class AliveExample extends Thread {  
    public void run() {  
        try {  
            Thread.sleep(1000);  
        } catch (InterruptedException e) {  
            System.out.println(e);  
        }  
    }  
}  
  
public static void main(String args[]) {  
    AliveExample t1 = new AliveExample();  
    t1.start();  
    System.out.println("Is thread t1 alive? " + t1.isAlive());  
    try {  
        t1.join(); // Wait for t1 to finish  
    } catch (InterruptedException e) {  
        System.out.println(e);  
    }  
    System.out.println("Is thread t1 alive after completion? " + t1.isAlive());  
}  
}
```

Output:

Is thread t1 alive? true

Is thread t1 alive after completion? false

6. `setName()` and `getName()` Methods

The setName() method sets a custom name for a thread, and getName() retrieves it.

Example:

```
class NameExample extends Thread {  
  
    public void run() {  
  
        System.out.println("Thread Name: " + Thread.currentThread().getName());  
  
    }  
  
    public static void main(String args[]) {  
  
        NameExample t1 = new NameExample();  
  
        t1.setName("MyThread");  
  
        t1.start();  
  
        System.out.println("Thread name set to: " + t1.getName());  
  
    }  
}
```

Output:

Thread name set to: MyThread

Thread Name: MyThread

7. setPriority() and getPriority() Methods

The setPriority(int value) method sets the priority of a thread, and getPriority() retrieves it. Thread priority ranges from 1 (MIN_PRIORITY) to 10 (MAX_PRIORITY), with 5 (NORM_PRIORITY) as the default.

Example:

```
class PriorityExample extends Thread {  
  
    public void run() {  
  
        System.out.println(Thread.currentThread().getName() + " Priority: " +  
Thread.currentThread().getPriority());  
  
    }  
}
```

```

public static void main(String args[]) {

    PriorityExample t1 = new PriorityExample();

    PriorityExample t2 = new PriorityExample();

    t1.setPriority(Thread.MIN_PRIORITY); // Priority 1

    t2.setPriority(Thread.MAX_PRIORITY); // Priority 10

    t1.start();

    t2.start();

}
}

```

Output:

Thread-0 Priority: 1

Thread-1 Priority: 10

(Note: Thread execution order may vary.)

8. yield() Method

The yield() method pauses the execution of the current thread and allows other threads of the same priority to execute.

Example:

```

class YieldExample extends Thread {

    public void run() {

        for (int i = 1; i <= 3; i++) {

            System.out.println(Thread.currentThread().getName() + " running...");

            Thread.yield(); // Suggests giving CPU control to other threads

        }

    }

}

public static void main(String args[]) {

```

```

        YieldExample t1 = new YieldExample();

        YieldExample t2 = new YieldExample();

        t1.start();

        t2.start();

    }

}

```

Java Runnable Interface

java.lang.Runnable is an interface that is to be implemented by a class whose instances are intended to be executed by a thread. There are two ways to start a new Thread – Subclass Thread and implement Runnable. There is no need to subclass a Thread when a task can be done by overriding only the run() method of Runnable.

Steps to Create a New Thread using Runnable

- Create a Runnable implementer and implement the run() method.
- Instantiate the Thread class and pass the implementer to the Thread, Thread has a constructor which accepts Runnable instances.
- Invoke start() of Thread instance, start internally calls run() of the implementer.
- Invoking start() creates a new Thread that executes the code written in run().
- Calling run() directly doesn't create and start a new Thread, it will run in the same thread.
- To start a new line of execution, call start() on the thread.

Runnable Interface Implementation

```

public class Java

{

    private class RunnableImpl implements Runnable

    {

        // Overriding the run Method

        @Override

        public void run()

        {

            System.out.println(Thread.currentThread().getName()+"executing run() method!");

        }

    }

}

```

```

    }

    // Main Method

    public static void main(String[] args)

    {

        System.out.println("Main thread is: " + Thread.currentThread().getName());

        // Creating Thread

        Thread t1 = new Thread(new Geeks().new RunnableImpl());

        // Executing the Thread

        t1.start();

    }

}

```

Output

Main thread is: main

Thread-0, executing run() method!

Thread Synchronization

Introduction to Thread Synchronization

Thread synchronization in Java ensures that multiple threads do not access shared resources at the same time, preventing race conditions and maintaining data consistency. Java provides built-in mechanisms like synchronized methods, synchronized blocks, static synchronization, and locks to handle synchronization.



Why Synchronization is Needed?

Without synchronization, multiple threads can access shared resources simultaneously, leading to inconsistent results and race conditions

Ways to Achieve Synchronization in Java

1. Synchronized Method – The entire method is locked so that only one thread can execute it at a time.

```
synchronized void method()
```

```
{
    // Critical section
}
```

2. **Synchronized Block** – Only a specific part of the method is locked, improving performance.

```
void method()
{
    synchronized(this)
    {
        // Critical section
    }
}
```

3. **Static Synchronization** – Used when working with static variables. Locks at the class level.

```
synchronized static void method()
{
    // Critical section
}
```

4. **Lock Interface (ReentrantLock)** – Provides more flexibility and control over synchronization.

```
ReentrantLock lock = new ReentrantLock();
lock.lock();
try {
    // Critical section
} finally {
    lock.unlock();
}
```

Collections

Introduction to Java Collections

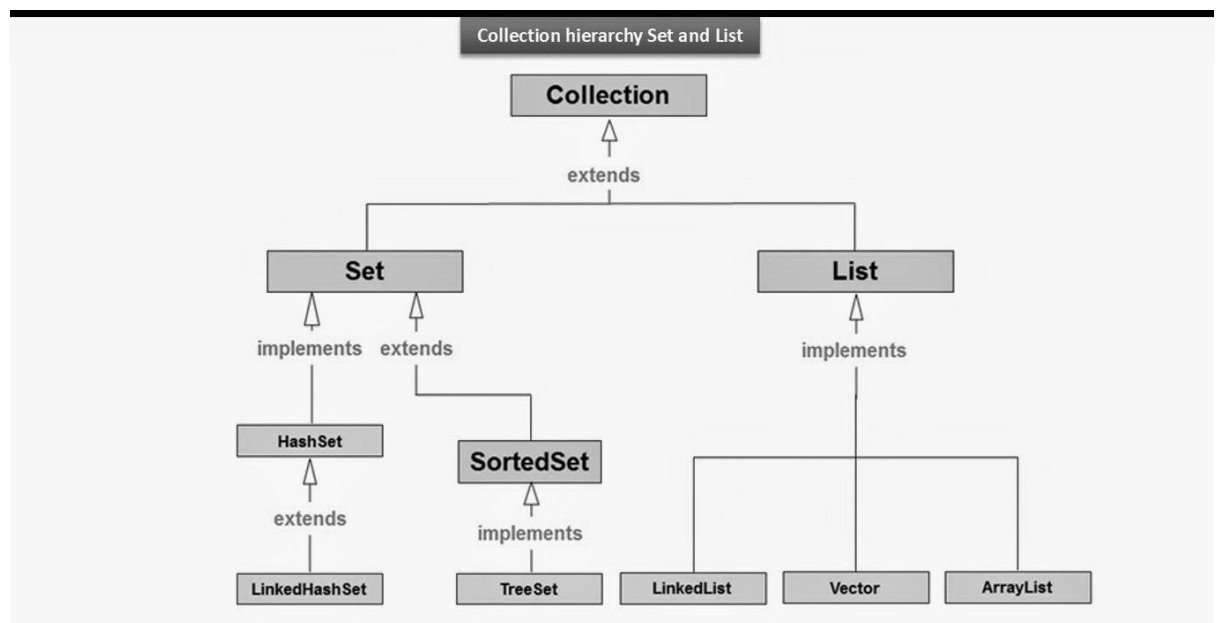
The Java Collections Framework (JCF) is a set of classes and interfaces that helps in storing, manipulating, and processing data efficiently. It provides various data structures like List, Set, Queue, and Map that allow dynamic storage and management of objects.

Advantages of Java Collections

- Reduces programming effort: Predefined classes like ArrayList, HashMap, etc., handle complex data structures.
- Improves performance: Optimized algorithms for searching, sorting, and iteration.
- Increases reusability: Can be used across different applications.
- Ensures type safety: Uses Generics to prevent type mismatches.

Collection Hierarchy in Java

The Collection framework consists of various interfaces and their implementations.



1. List Interface

A List is an ordered collection that allows duplicate elements and provides indexed access.

1.1 ArrayList

ArrayList is a resizable array that allows fast random access but slow insertion/deletion.

Example:


```
import java.util.ArrayList;

public class ArrayListExample {

    public static void main(String[] args) {

        ArrayList<String> list = new ArrayList<>();

        list.add("Apple");

        list.add("Banana");

        list.add("Orange");

        System.out.println(list); // Output: [Apple, Banana, Orange]

    }

}
```

1.2 LinkedList

LinkedList is a doubly linked list that allows fast insertion/deletion but slow random access.

Example:

```
import java.util.LinkedList;

public class LinkedListExample {

    public static void main(String[] args) {

        LinkedList<String> list = new LinkedList<>();

        list.add("Dog");

        list.add("Cat");

        list.addFirst("Elephant");

        System.out.println(list); // Output: [Elephant, Dog, Cat]

    }

}
```

1.3 Vector

Vector is similar to ArrayList but is thread-safe (synchronized).

Example:

```
import java.util.Vector;

public class VectorExample {

    public static void main(String[] args) {

        Vector<Integer> vec = new Vector<>();

        vec.add(10);

        vec.add(20);

        vec.add(30);

        System.out.println(vec); // Output: [10, 20, 30]

    }

}
```

1.4 Stack

Stack is a LIFO (Last-In-First-Out) data structure.

Example:

```
import java.util.Stack;

public class StackExample {

    public static void main(String[] args) {

        Stack<String> stack = new Stack<>();

        stack.push("A");

        stack.push("B");

        stack.push("C");

        System.out.println(stack.pop()); // Output: C

    }

}
```

2. Set Interface

A Set does not allow duplicate elements and is unordered (except TreeSet).

2.1 HashSet

HashSet stores unique elements using a hash table and is unordered.

Example:

```
import java.util.HashSet;

public class HashSetExample {

    public static void main(String[] args) {

        HashSet<Integer> set = new HashSet<>();

        set.add(10);

        set.add(20);

        set.add(10); // Duplicate, will be ignored

        System.out.println(set); // Output: [10, 20]

    }

}
```

2.2 LinkedHashSet

LinkedHashSet maintains insertion order while storing unique elements.

Example:

```
import java.util.LinkedHashSet;

public class LinkedHashSetExample {

    public static void main(String[] args) {

        LinkedHashSet<String> set = new LinkedHashSet<>();

        set.add("One");

        set.add("Two");

        set.add("Three");

        System.out.println(set); // Output: [One, Two, Three]

    }

}
```

2.3 TreeSet

TreeSet stores elements in sorted order.

Example:

```
import java.util.TreeSet;

public class TreeSetExample {

    public static void main(String[] args) {

        TreeSet<Integer> set = new TreeSet<>();

        set.add(50);

        set.add(30);

        set.add(40);

        System.out.println(set); // Output: [30, 40, 50]

    }

}
```

3. Queue Interface

A Queue follows FIFO (First-In-First-Out) order.

3.1 PriorityQueue

PriorityQueue orders elements based on priority.

Example:

```
import java.util.PriorityQueue;

public class PriorityQueueExample {

    public static void main(String[] args) {

        PriorityQueue<Integer> pq = new PriorityQueue<>();

        pq.add(30);

        pq.add(10);

        pq.add(20);

        System.out.println(pq.poll()); // Output: 10 (smallest element first)

    }

}
```

```
}  
  
}
```

3.2 Deque (ArrayDeque)

Deque allows insertion and removal from both ends.

Example:

```
import java.util.ArrayDeque;  
  
public class ArrayDequeExample {  
  
    public static void main(String[] args) {  
  
        ArrayDeque<String> deque = new ArrayDeque<>();  
  
        deque.addFirst("Start");  
  
        deque.addLast("End");  
  
        System.out.println(deque); // Output: [Start, End]  
  
    }  
  
}
```

4. Map Interface

A Map stores key-value pairs and does not allow duplicate keys.

4.1 HashMap

HashMap stores key-value pairs unordered.

Example:

```
import java.util.HashMap;  
  
public class HashMapExample {  
  
    public static void main(String[] args) {  
  
        HashMap<Integer, String> map = new HashMap<>();  
  
        map.put(1, "Java");  
  
        map.put(2, "Python");  
  
        System.out.println(map); // Output: {1=Java, 2=Python}
```

```
}  
  
}
```

4.2 LinkedHashMap

LinkedHashMap maintains insertion order.

Example:

```
import java.util.LinkedHashMap;  
  
public class LinkedHashMapExample {  
  
    public static void main(String[] args) {  
  
        LinkedHashMap<Integer, String> map = new LinkedHashMap<>();  
  
        map.put(1, "A");  
  
        map.put(2, "B");  
  
        System.out.println(map); // Output: {1=A, 2=B}  
  
    }  
  
}
```

4.3 TreeMap

TreeMap stores elements in sorted order of keys.

Example:

```
import java.util.TreeMap;  
  
public class TreeMapExample {  
  
    public static void main(String[] args) {  
  
        TreeMap<Integer, String> map = new TreeMap<>();  
  
        map.put(3, "Three");  
  
        map.put(1, "One");  
  
        map.put(2, "Two");  
  
        System.out.println(map); // Output: {1=One, 2=Two, 3=Three}  
  
    }  
  
}
```

```
}
```

Introduction to JavaBeans

JavaBeans are reusable software components for Java that can be manipulated visually in a builder tool. A JavaBean is a simple Java class that follows certain conventions: it should be serializable, have a no-argument constructor, and use getter and setter methods to access properties (also called encapsulation). JavaBeans are used in GUI development, frameworks like JSP, and enterprise applications for maintaining reusability and modularity.

Key features of JavaBeans:

- Must have a public no-arg constructor
- Must be serializable
- Properties must be private with public getter/setter

Example of a JavaBean:

```
import java.io.Serializable;

public class StudentBean implements Serializable
{
    private String name;

    private int age;

    public StudentBean() {} // No-arg constructor

    public String getName()
    {
        return name;
    }

    public void setName(String name)
    {
        this.name = name;
    }

    public int getAge()
```

```

{
    return age;
}

public void setAge(int age)
{
    this.age = age;
}
}

```

Introduction to Network Programming

Network programming in Java is used to write programs that can communicate with other machines over a network (like the Internet or a LAN). Java provides the `java.net` package which includes classes like `Socket`, `ServerSocket`, `InetAddress`, and `URL` to handle network tasks such as creating servers, clients, and reading data over the internet.

1. Server (Listens for client messages):

```

import java.io.*;

import java.net.*;

public class ServerExample
{
    public static void main(String[] args) throws IOException
    {
        ServerSocket server = new ServerSocket(5000);

        Socket socket = server.accept();

        DataInputStream dis = new DataInputStream(socket.getInputStream());

        String message = dis.readUTF();

        System.out.println("Client says: " + message);

        server.close();
    }
}

```



```
}
```

2. Client (Sends message to server):

```
import java.io.*;
```

```
import java.net.*;
```

```
public class ClientExample
```

```
{
```

```
    public static void main(String[] args) throws IOException
```

```
{
```

```
    Socket socket = new Socket("localhost", 5000);
```

```
    DataOutputStream dos = new DataOutputStream(socket.getOutputStream());
```

```
    dos.writeUTF("Hello Server!");
```

```
    socket.close();
```

```
}
```

```
}
```



SAHYADRI .